ILCC

Instant Loan on Credit Card: 4 web APIs plus the NTB batch process

One guide that explains how a loan request travels from the channel app, through the ILCC web services, into Prime, and then (for new-to-bank customers) through the batch jobs and IBM MQ to the payment system.

Written from the ILCC web code and the batch code (ILCC_NTB, ILCC_NTB_OUTQUEUE, KAFKA_CONSUMER). Where the supplied code uses mock responses or has an issue, this guide says so.

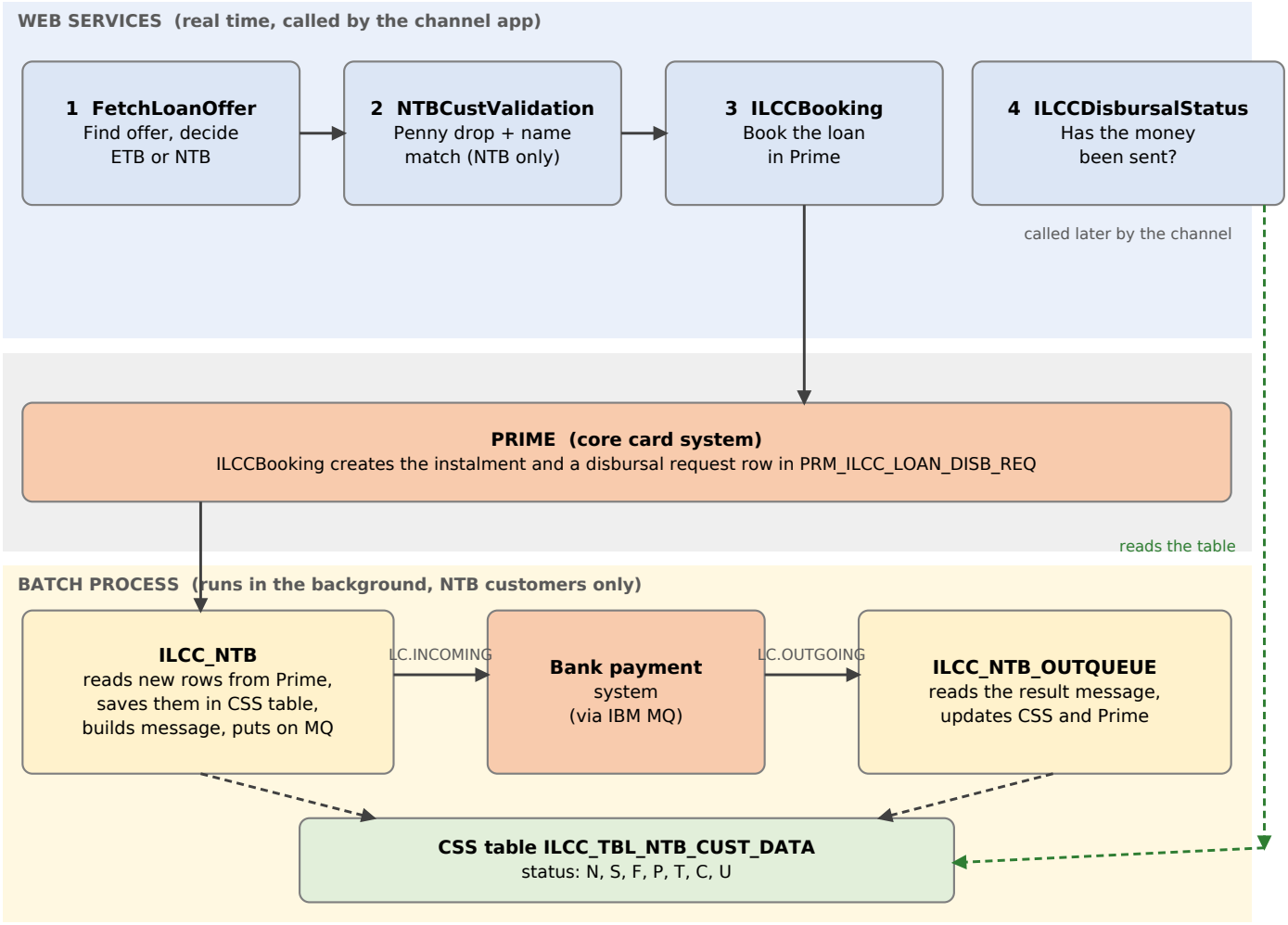
1. ILCC in plain words

A cardholder can turn part of the credit-card limit into an instant loan (EMI). A channel app (mobile banking, internet banking and similar) calls four ILCC web APIs, in order: **find the offer, verify the bank account (new customers only), book the loan, check the disbursal**.

Where the money goes depends on the customer type. An **ETB** (existing-to-bank) customer gets the money in their own Axis account, and Prime handles it. An **NTB** (new-to-bank) customer gets it by NEFT in another bank. That payment is sent by the **batch process** through IBM MQ, which is why the batch jobs are part of this project.

Term	Meaning in this project
ETB / NTB	Existing-to-bank / New-to-bank customer. Decided by looking up the customer in Finacle (account found = ETB).
WTL, OTL	**Within the Limit** and **Over the Limit** loans (the names are spelled out in the DigitalLending code). Asked from Prime as PLCC-WTL and PLCC-OTL.
CardSerNo	Internal serial number of the card in Prime. Main key used by all four APIs.
ReferenceId	Unique id of each API call. The same id sent twice is rejected (code 101).
TokenRefId	Id of one customer journey. Created by FetchLoanOffer and re-used by the next steps.
Prime	The core credit-card system. ILCC reads offers and creates the instalment there.
ESB	Bank integration layer. Most calls go through it as AES-encrypted JSON.
Penny drop (IMPS)	A tiny test transfer to a bank account. The bank returns the account holder name, which proves the account exists.
Posidex	Name-matching service. Compares the card holder name with the penny-drop name.
IBM MQ	Message queue used to send the NTB payment instruction and to receive the result.
EPH status	Payment result code returned on the outgoing queue (ACSC, RJCT, and so on). Table in section 6.
CSS	The database that holds the ILCC logs, master rows and batch tables.

2. The whole journey at a glance



Read the diagram like this

- **Steps 1 to 3** are one customer journey. The channel keeps the same TokenRefId for all three.
- **Step 2 is skipped for ETB customers.** They pick one of their own Axis accounts in step 3.
- **ILCCBooking does not move money itself.** It asks Prime to create the instalment. For NTB, Prime writes a disbursal request, and the batch jobs pick it up from there.
- **Step 4 reads what the batch jobs wrote.** If the batch is not running, an NTB loan stays at "Not Disbursed".

3. Rules common to all four web APIs

Every API follows the same skeleton before and after its own logic:

WEB step 1	The request arrives as an encrypted JSON body. It is decrypted and the service name is read from `Request.Body`.
CHECK step 2	**Authenticate the channel.** Allowed channels for ILCC: **Test, Flipkart CO, MBB, IBB, SOB, SIDDHI, SAK**. Wrong channel or password gives 001 or 002.
CHECK step 3	**Validate each field** with a pattern (section 4 lists them). A bad value gives the matching code, for example 007 invalid IFSC.
CHECK step 4	**Reject a repeated ReferenceId** (code 101) by looking in ILCC_Tbl_Request_Details.
CHECK step 5	**Check the token** according to the API (table below).
DB step 6	**Log everything**: one row in ILCC_Tbl_Request_Details (masked request, response, time, CUST_TYPE, TOKEN_REF_ID) and one row per external call in ILCC_Tbl_External_Api_log.
RESULT step 7	Build the response and encrypt it.

How the token (TokenRefId) is checked

API	Token rule	Error if the rule fails
FetchLoanOffer	Token must be **new**. It is the start of the journey.	103 Token Id Already Exist
NTBCustValidation	Token must **already exist** (created by FetchLoanOffer, within the last 2 days).	004 / token validation failed
ILCCBooking	Checked with its own booking rule against the earlier steps.	token validation failed
ILCCDisbursalStatus	No token. Uses CardSerNo and INSTLRefNo.	-

4. The web APIs one by one

4.1 FetchLoanOffer: find the loan offer

What it does: tells the channel whether the card has a loan offer, which offers (WTL, OTL), and whether the customer is ETB or NTB. It also opens the journey: it creates the master row that the next steps use.

Field	Required	Format (pattern from the code)
ReferenceId	Mandatory	`^[0-9]{15,20}$`
TokenRefId	Mandatory	`^[0-9a-zA-Z]{15,20}$`
CardSerNo	Mandatory	`^[0-9]{5,16}$`
CardNo	Optional	`^[0-9]{15,16}$`
CustType	Optional	`^[eEnN][tT][bB]$`
FirstName	Optional	`^[a-zA-Z][a-zA-Z\s]{2,30}$`
LastName	Optional	`^[a-zA-Z]{2,30}$`

CardNo, CustType, FirstName and LastName are optional as a group: if CardNo is sent, the other three become mandatory (else code 106).

Business flow

WEB step 1	Common checks (section 3). The token must be new.
CHECK step 2	**Was CardNo sent?** Yes: go to step 3. No: go to step 4.
CALL step 3	**Path A, CardNo sent.** Ask Prime two eligibility questions with the card number: one for **PLCC-WTL** and one for **PLCC-OTL** (EligibilityCheckInquiry via ESB). Prime returns code 10 = not found, 0 = offer available, other = not available.
CALL step 4	**Path B, only CardSerNo sent.** (a) CreditCardESBService converts the serial number into the card number. (b) Prime EntityInquiry returns the customer id, account number and names. (c) At the same time, Finacle get-account-details decides ETB or NTB, and Prime is asked the WTL and OTL eligibility questions.
CHECK step 5	**ETB or NTB (path B).** Finacle returns accounts with a match: customer is **ETB** and the masked accounts are returned in `CASADetails.AccNo`. No record or no match: **NTB**. Anything else: external API failure.
CHECK step 6	**Build the offer answer.** WTLOfferAvailFlag and OTLOfferAvailFlag are Y or N. The Description says: "Loan Offers are Available", "WTL offer is not Available", "OTL offer is not Available" or "Loan Offers are not Available". Each available offer returns details such as instalment type and foreclosure fee.
DB step 7	On success, **save a master row** in Ilcc_Tbl_Temp_Master_Details: REF_ID, TOKEN_REF_ID, CARD_SER_NO, CUST_TYPE, card number, encrypted account number, customer id and names.
RESULT step 8	Return CardSerno, CustType, CASADetails and OfferDetls (OfferDtls list).

Result codes: 000 success, 102 record not found, 103 token already used, 106 mandatory value missing, 222 external API failure, 999 technical error.

Watch out (supplied code). In Path B, `getcardno` returns a plain card number or the word "Failure", but the caller treats the result as a dictionary (`.get("status")`), and the next step calls a number as if it were a function. As written, **Path B cannot succeed** and ends in 222. Path A is the one that works. Path A also uses mock offer responses (`get_offer_wtl_res`, `get_offer_otl_res`) instead of the live Prime reply.

4.2 NTBCustValidation: check the bank account (NTB only)

What it does: proves that the NTB customer's other-bank account exists and belongs to the same person, before any money is sent.

Field	Required	Format (pattern from the code)
ReferenceId	Mandatory	`^[0-9]{15,20}$`
TokenRefId	Mandatory	`^[0-9a-zA-Z]{15,20}$`
CardSerNo	Mandatory	`^[0-9]{5,16}$`
AccNo	Mandatory	`^[0-9]{8,20}$`
IFSC	Mandatory	`^[0-9a-zA-Z]{9,15}$`

Business flow

WEB step 1	Common checks. The token must already exist (created by FetchLoanOffer).
DB step 2	Read the master row by TokenRefId (only rows from the last 2 days). It holds the customer first and last name.
CALL step 3	**IMPS penny drop** to AccNo + IFSC through the internal IMPS service. The bank answers PennyDropStatus Y or N and the beneficiary name (BenName).
CHECK step 4	**Penny drop failed?** Return PennyDropStatus N and NameMatchStatus N. The name check is not attempted.
CALL step 5	**Posidex name match** through ESB: card holder name against BenName. Posidex returns matchPercentage.
CHECK step 6	**Decide.** 75 percent or more: NameMatchStatus **Y**. Below 75: **N**. Empty percentage while a name exists: external API failure.
DB step 7	Update the master row with the account details (encrypted) and log both external calls.
RESULT step 8	Return CardSerNo, PennyDropStatus and NameMatchStatus.

Watch out (supplied code). The IMPS and Posidex replies come from mock variables (`imps_resp`, `pos_response`), not from live calls. The 75 percent threshold is hardcoded.

4.3 ILCCBooking: book the loan

What it does: picks the instalment plan in Prime and creates the instalment (the loan). The channel sends the amount, tenure, rate and fees that the customer accepted.

Field	Required	Format (pattern from the code)
ReferenceId	Mandatory	`^[0-9]{15,20}$`
TokenRefId	Mandatory	`^[0-9a-zA-Z]{15,20}$`
CardSerNo	Mandatory	`^[0-9]{5,16}$`
AccNo	Mandatory	`^[0-9][0-9X]{7,20}$`
Tenure	Mandatory	`^[0-9]{1,4}$`
LoanAmnt	Mandatory	`^[0-9]{1,10}$`
InterestRate	Mandatory	`^[0-9.]{1,5}$`
ProcFeeFlat	Mandatory	`^[0-9.]{1,6}$`
FCFeeFlat	Mandatory	`^[0-9.]{1,6}$`
ProcFeePer	Mandatory	`^[0-9.]{1,3}$`
FCFeePer	Mandatory	`^[0-9.]{1,3}$`
INSTLType	Mandatory	`^[a-zA-Z]{3,5}$`

Business flow

WEB step 1	Common checks and token check.
DB step 2	Read the master row (card number, customer type, stored accounts). **ETB without AccNo** stops here with "AccNo Empty".
DB step 3	For NTB, look up the customer bank name from the IFSC in ilcc_tbl_bank_mapping_conf. If not found, the name defaults to "Axis Bank". Read the current Prime ticket from Tbl_Ticket_Request_Details.
CALL step 4	**Prime GetInstalmentPlanInquiry** with card number, tenure, amount, rate, fees and instalment type. Prime returns the plan code. Code 4 means no plan found (102). Any other non-zero code is an external failure (222).
CHECK step 5	**How is the money paid?** ETB: **Account Transfer** to the chosen own account (the stored account matching the AccNo the caller sent). NTB: **NEFT** to the customer's other bank.
CALL step 6	**Prime InstalmentCreation** with the plan code and the payment details. Code 12 means not found. Zero means the instalment exists.
DB step 7	Save the amount and BOOKING_STATUS on the master row. Log the external calls.
RESULT step 8	Return CardSerNo, Status Y or N, INSTLRefNo (the instalment reference, which is the key for step 4) and a description.

Watch out (supplied code). Both Prime replies come from mock variables (`plan_inquiry_resp`, `plan_creation_resp`). The live InstalmentCreation call is disabled. Before go-live these must be switched to the real ESB calls.

4.4 ILCCDisbursalStatus: has the money been sent?

What it does: tells the channel the state of the loan payment. It reads a different place for NTB and ETB.

Field	Required	Format (pattern from the code)
ReferenceId	Mandatory	`^[0-9]{15,20}$`
CardSerNo	Mandatory	`^[0-9]{5,16}$`
INSTLRefNo	Mandatory	`^[0-9]{5,20}$`

Business flow

WEB step 1	Common checks. No token is needed.
DB step 2	Read CUST_TYPE from the master row using CardSerNo.
DB step 3	**NTB:** read ILCC_TBL_NTB_CUST_DATA where INSTALLMENT_NO = INSTLRefNo. This table is kept up to date by the batch jobs. The status letter is turned into text (table below).
DB step 4	**ETB:** read Prime table prm_loan_disb_req where REQUESTID = INSTLRefNo (through the Prime connection). Code 000 or Success = Y, anything else = N. The description is "Disbursal Success" or "Disbursal Failure".
RESULT step 5	Return DisbursalStatus, INSTLRefNo and Description. If no row is found: 102.

Status letter (NTB)	Text returned in Description	DisbursalStatus
N	Not Disbursed (waiting to be sent)	N
R	Not Disbursed	N
S	Not Disbursed (sent to MQ, no result yet)	N
P	Disbursal in Progress	Y
T	Disbursal Sent	Y
C	Disbursal Successful	Y
F	Disbursal Failed	Y
U	Disbursal Failed (payment returned)	Y

Important for the channel team. DisbursalStatus is **N only for N, R and S**. For every other letter (including F and U, which are failures) it is **Y**. The real outcome is in **Description**, so the channel should read the text, not only the Y or N flag.

5. The NTB batch process

Three jobs form the NTB payment pipeline. They are registered in the batch Config.py as **ILCC_NTB**, **ILCC_NTB_OUTQUEUE** and **KAFKA_CONSUMER** (plus ILCC_NTB_PAT for the PAT test environment). The only link between them and the web APIs is the data in the tables.

Job	Python file used (batch Config.py)	Does what
ILCC_NTB	ILCC_NTB_Core_Integration_SSL.py	Takes new loan payment rows, sends them to the bank through MQ, reads the acknowledgement.
ILCC_NTB_OUTQUEUE	ILCC_NTB_Outqueu_Core_Integration_SSL.py	Reads the payment result from MQ and updates CSS and Prime.
KAFKA_CONSUMER	Kafka_Consumer_core_process_Karthik.py	Reads loan messages from Kafka topic css-prime-disbursement-ack and inserts them into the same CSS table (second way to feed the pipeline).
ILCC_NTB_PAT	(PAT variant)	Same as ILCC_NTB, but against the PAT Prime database.

5.1 Job ILCC_NTB: send the payment instruction

BATCH step 1	**Find the newest instalment already stored** in CSS (the highest INSTALLMENT_NO).
DB step 2	**Read new rows from Prime** table PRM_ILCC_LOAN_DISB_REQ where REQUESTID is greater than that number and RESP_CODE is empty. Each row has amount, customer name, beneficiary account, beneficiary name, IFSC and the transfer type. Email and mobile come from Prime functions through the DB link.
DB step 3	**Save them** in CSS table ILCC_TBL_NTB_CUST_DATA with status **N**. The beneficiary account is stored encrypted.
CHECK step 4	**No new rows?** Skip to step 8 (still collect any waiting acknowledgements).
DB step 5	**Select the rows with status N** from ILCC_TBL_NTB_CUST_DATA.
MQ step 6	**Build the XML payment instruction** from the incoming-message template and **put it on queue LC.INCOMING** with a unique REF_ID. Wait 5 seconds between messages.
DB step 7	**Log the message** in ILCC_TBL_MESSAGE_DETAILS (REF_ID, message, time, INSTALLMENT_NO, status N).
MQ step 8	**Read the acknowledgement queue LC.ACKNOWLEDGEMENT.** The message has MsgRef (the REF_ID) and AckResponse.
CHECK step 9	**AckResponse is "Sent for Processing"?** Yes: store the ack in ILCC_TBL_MESSAGE_DETAILS and set the customer row to **S**. No: set the customer row to **F**.

5.2 Job ILCC_NTB_OUTQUEUE: read the payment result

This job runs in a loop and waits for messages on queue **LC.OUTGOING**.

MQ step 1	**Get a message** from LC.OUTGOING and read three fields: the payment status (EPH status), a description and the reference id (REF_ID).
CHECK step 2	**Translate the EPH status** with the table below. This gives the status letter, the settlement status and the response code.
DB step 3	**Update the customer row** in ILCC_TBL_NTB_CUST_DATA: status letter and EPH_Status, where the instalment number is the one logged for that REF_ID.
DB step 4	**Update the message log** ILCC_TBL_MESSAGE_DETAILS with the outgoing message and its time.
DB step 5	**Update Prime** table PRM_ILCC_LOAN_DISB_REQ (TRANSACTION_STATUS, SETTLEMENT_STATUS, RESP_CODE) for that REQUESTID, so Prime also knows the result.

EPH status in the message	Status letter in CSS	Settlement status	Response code	What the web API shows
ACSC	C	Success	Success	Disbursal Successful
OSNT	P	-	Processing	Disbursal in Progress
ACAR	T	-	Success	Disbursal Sent
RJCT	F	-	Failure	Disbursal Failed
RTRN	U	FailReturn	Failure	Disbursal Failed (returned)

5.3 Job KAFKA_CONSUMER: second input path

It listens on Kafka topic **css-prime-disbursement-ack** (broker 10.9.205.24:9092, SASL) and inserts each JSON message into ILCC_TBL_NTB_CUST_DATA (instalment number, total amount, remitter and beneficiary fields, bank reference, batch id). The row then continues through ILCC_NTB as above. The Kafka-based variants of ILCC_NTB and OUTQUEUE (names ending Kafka_Core) are commented out in Config.py and are not active.

MQ connection details

Item	Value
Queue manager / channel	IIBQMAGWT01, channel LC, SSL cipher ECDHE_RSA_AES_128_GCM_SHA256
Host	esbmquatclient.axisb.com, port 1516
Queues	Incoming LC.INCOMING, Acknowledgement LC.ACKNOWLEDGEMENT, Outgoing LC.OUTGOING
Config	Conf/mqueue.ini, Lib/Mq_Interface_SSL.py (credentials not shown)

6. How the web APIs and the batch jobs connect

Step	Who does it	Reads	Writes
Offer	FetchLoanOffer (web)	Prime offer + Finacle	Ilcc_Tbl_Temp_Master_Details
Account check (NTB)	NTBCustValidation (web)	master row, IMPS, Posidex	master row
Book	ILCCBooking (web)	master row, Prime plan	Prime instalment, master row (BOOKING_STATUS)
Pick up	ILCC_NTB (batch)	Prime PRM_ILCC_LOAN_DISB_REQ	ILCC_TBL_NTB_CUST_DATA (N)
Send	ILCC_NTB (batch)	ILCC_TBL_NTB_CUST_DATA (N)	MQ LC.INCOMING, ILCC_TBL_MESSAGE_DETAILS, status S or F
Result	ILCC_NTB_OUTQUEUE (batch)	MQ LC.OUTGOING	status C, P, T, F or U, and Prime PRM_ILCC_LOAN_DISB_REQ
Show	ILCCDisbursalStatus (web)	ILCC_TBL_NTB_CUST_DATA (NTB) or prm_loan_disb_req (ETB)	request log

Timeline of one NTB loan: book (status does not exist yet) → ILCC_NTB picks it up (**N**) → sent to MQ and acknowledged (**S**) → bank result arrives (**P**, **T**, then **C**, or **F**/**U**).

7. Reference

External systems used by the ILCC web APIs

Call	URL (UAT)	Used by	State in supplied code
Prime EntityInquiry	https://esbuat2.axisb.com/json/v2/prime4_0axisissuerapp/EntityInquiry	FetchLoanOffer (path B)	Live call
CreditCardESBService	https://esbuat2.axisb.com/v2/JSON/CreditCardESBService	FetchLoanOffer (path B)	Live call
Finacle account details	https://esbuat2.axisb.com/finacle/account-details/enc/get-account-details	FetchLoanOffer (ETB/NTB)	Live call

Call	URL (UAT)	Used by	State in supplied code
Prime EligibilityCheckInquiry	https://esbuat2.axisb.com/json/v2/prime4_0axisapp/EligibilityCheckInquiry	FetchLoanOffer (WTL, OTL)	Live in path B, mock in path A
IMPS fund transfer	http://192.168.12.85:8200/api/v1/ImpsServices	NTBCustValidation	Mock reply
Posidex name match	https://esbuat2.axisb.com/posidex/customernamematch/enc/match	NTBCustValidation	Mock reply
Prime GetInstalmentPlanInquiry	https://esbuat2.axisb.com/json/v2/prime4_0axisapp/GetInstalmentPlanInquiry	ILCCBooking	Mock reply
Prime InstalmentCreation	https://esbuat2.axisb.com/json/v2/prime4_0axisapp/InstalmentCreation	ILCCBooking	Disabled, mock reply

Database tables

Table	Used by	Purpose
ILCC_Tbl_Request_Details	All web APIs	One row per API call (duplicate check, audit)
ILCC_Tbl_External_Api_log	All web APIs	One row per external call (sent and received)
Ilcc_Tbl_Temp_Master_Details	Web APIs 1 to 4	Journey master row, keyed by TokenRefId
ilcc_tbl_bank_mapping_conf	ILCCBooking	IFSC to bank name
Tbl_Ticket_Request_Details	ILCCBooking (web)	Current Prime ticket, filled by the batch job TICKET_GEN
prm_loan_disb_req / PRM_ILCC_LOAN_DISB_REQ	ETB status (web), ILCC_NTB and OUTQUEUE (batch)	Prime disbursal request rows
ILCC_TBL_NTB_CUST_DATA	Batch jobs, NTB status (web)	NTB payment rows with status letter
ILCC_TBL_MESSAGE_DETAILS	Batch jobs	Each MQ message, its acknowledgement and result

Response codes (ILCC)

Code	Meaning	Code	Meaning
000	Request Succeeded	001	Authentication Failed
002	Invalid Channel Id	003	Invalid Reference Id
004	Invalid Token Reference Id	005	Invalid Card Ser No
006	Invalid Account No	007	Invalid IFSC Code
008	Invalid Instalment Type	009	Invalid Tenure
010	Invalid Loan Amount	011	Invalid Interest Rate
012	Invalid Processing Fee Flat	013	Invalid Foreclosure Fee Flat
014	Invalid Processing Fee Percent	015	Invalid Foreclosure Fee Percent
016	Invalid Instalment Ref number	017	Invalid Customer type
018	Invalid Card Number	019	Invalid FirstName
020	Invalid LastName	101	ReferenceId Already Exist
102	Record Not Found	103	Token Id Already Exist
104	The customer is neither ETB nor NTB	106	Mandatory values are missing
108	Invalid CustID	111	Service temporarily unavailable
222	External API Failure	999	Unable to Process, Technical Errors
999	Unable to Process, Technical Error		

8. Watch-outs and quick troubleshooting

Things found in the supplied code

- **Mock replies.** IMPS, Posidex, Prime plan inquiry and instalment creation (and offers on path A) use mock variables. Switch them to the real calls before production.
- **FetchLoanOffer with only CardSerNo cannot succeed** as written (type mismatch, section 4.1). Send CardNo, CustType, FirstName and LastName until it is fixed.
- **DisbursalStatus Y does not mean success.** Read Description (section 4.4).
- **Shared global dictionaries.** The offer step stores card, account and name in module-level dictionaries (`g_tempcustdetails` and the external-API detail dictionaries). Two simultaneous requests can overwrite each other.
- **Batch loops.** ILCC_NTB_OUTQUEUE runs a continuous loop; the core process re-runs a failed step every 10 seconds with no limit.
- **Many copies of the same script.** ILCC_NTB has 10 files in Bin, OUTQUEUE 7, KAFKA_CONSUMER 6. Only the files named above are active. The sql_transactions*.ini backups also hold old queries; the live one is in sql_transactions.ini.
- **Secrets.** Config files hold passwords and keys in plain text (masked in this guide).

If something looks wrong

What you see	Where to look first
FetchLoanOffer returns 103	The TokenRefId was used before. Generate a new token for a new journey.
NTBCustValidation says token validation failed	FetchLoanOffer was not called first with this token, or it is older than 2 days.
ILCCBooking returns "AccNo Empty"	Customer is ETB and AccNo was not sent.
NTB status stays "Not Disbursed" with letter N	ILCC_NTB is not running, or Prime has not created the row (RESP_CODE must be empty).
Letter S for a long time	Message went to MQ but no result yet. Check ILCC_NTB_OUTQUEUE and queue LC.OUTGOING; check ILCC_TBL_MESSAGE_DETAILS for the REF_ID.
Letter F	Acknowledgement was not "Sent for Processing", or the bank returned RJCT. Read EPH_Status on the row.
Letter U	Payment was sent and then returned (RTRN).